

Research Methodology

Prinz Edward L. Ducyogen

Centro Escolar University

Course Title

Professor Name

Due Date

Author Note

Correspondence concerning this article should be addressed to Prinz Edward L.

Ducyogen, Email: ducyogenpred@gmail.com

Research Methodology

Chapter 1 established the research gap, the research questions, and the objectives of the study. This chapter translates those commitments into a replicable research plan. The chapter specifies the research framework, the subjects of the study, the sampling technique, and the instruments. It then defines the data gathering procedure, the statistical treatment, and the data analysis. The chapter closes with the software system description and the testing and evaluation protocol.

3.1 Research Methodology

This study adopts the Design Science Research Methodology (DSRM) of Peffers et al. (2007) as its governing framework. The framework directs research through the systematic design, development, demonstration, and evaluation of computational artifacts. It defines six activities, from problem identification and motivation to communication. Each activity produces defined outputs that feed the next activity. The sequence also permits iteration, because evaluation results can trigger renewed design and development. This study enters the sequence at the first activity, the problem-centered entry point, because Chapter 1 derives the objectives from an observed research gap.

DSRM fits this study for three reasons. First, the study creates designed artifacts, which is the defining work of design science. The artifacts are the Multi-Task Corpus of Taglish Disaster Tweets, a reproducible training and benchmarking pipeline, and a prototype triage application. Second, the research questions motivate controlled comparison of optimization configurations on those artifacts. Quantitative experimental benchmarking therefore serves as the evaluation strategy inside the DSRM lifecycle.

Third, the study aims at knowledge for practice under local deployment constraints.

Design science sets the same requirement, because an artifact earns acceptance by solving an important problem in a host setting ([Hevner et al., 2004](#)).

Chapter 1 organizes the six activities into four phases, namely Baseline Analysis, Design and Artifact Creation, Empirical Benchmarking, and Comparative Validation. The problem identification and objectives activities are complete in Chapter 1. The design, demonstration, evaluation, and communication activities unfold across the phases and the manuscript as a whole.

The demonstration and evaluation activities each span two phases. The baseline analysis demonstrates and evaluates current practice before the study designs its own artifact. The benchmarking phase then demonstrates the candidate methods, and the validation phase completes the evaluation. Iteration connects the activities in practice. Baseline findings can motivate revisions to the corpus design or the training pipeline before the artifact work concludes.

The Input-Process-Output model in Chapter 1 restates the same lifecycle at the level of inputs, processes, and outputs. Section 3.2 defines the inputs as the subjects of the study. Sections 3.3 to 3.5 specify the processes for sampling, data gathering, and measurement. Section 3.8 describes the software artifact under design and development. Sections 3.6, 3.7, and 3.9 plan the statistical analysis and the evaluation of the outputs.

3.2 Subjects of the Study

Section 3.1 established the lifecycle of the study. This section defines the subjects that carry it. The study works on four subjects, the corpus, the encoder backbones, the

optimization configurations, and the prototype service.

Subject 1 is the Multi-Task Corpus of Taglish Disaster Tweets. The corpus targets 6,000 to 10,000 public posts from four typhoons, namely Haiyan, Vamco, Rai, and Paeng. The study sets 6,000 verified tweets as the minimum corpus size. Each tweet carries four annotation layers, a word-level language tag for every word, begin-inside-outside (BIO) entity spans for locations and infrastructure, and two sequence labels for intent and urgency. The intent label comes from a triage-oriented merge of the HumAID categories ([Alam et al., 2021](#)). The urgency label uses the four-level scale of Sarioglu Kayi et al. ([2020](#)), which converts to a binary urgent or not urgent value. The word-level tags follow the convention of TweetTaglish ([Herrera et al., 2022](#)).

Every corpus record is one JSON object in a JSONL file. Each object carries seven fields:

- `tweet_id`, a unique identifier for the tweet
- `event`, the typhoon that produced the tweet
- `text`, the raw message text
- `language_tags`, one tag per word, with English, Tagalog, mixed, ambiguous, and other as values ([Gambäck & Das, 2016](#); [Herrera et al., 2022](#))
- `bio_tags`, one BIO label per word, with LOC and INFRA as entity types and O as the outside label
- `intent`, one label from the merged HumAID taxonomy
- `urgency`, one label from the four-level scale of Sarioglu Kayi et al. ([2020](#))

Subject 2 covers the four candidate encoder backbones, XLM-RoBERTa Base,

multilingual BERT Base, SEA-LION ModernBERT, and RoBERTa-Tagalog Base. Each candidate places three linear heads on the shared backbone under hard parameter sharing. One token head decodes entity labels, and two sequence heads decode intent and urgency. The candidates first undergo fine-tuning under uniform equal weighting across the leave-one-event-out (LOEO) folds. The study then selects one backbone by mean macro F1 across folds for the full optimization benchmark. The heads are small linear layers beside the shared backbone, so inference time stays close to single-encoder cost.

Subject 3 is the optimization benchmark of 10 configurations. The treatment group pairs two families, dynamic loss weighting and gradient surgery. Dynamic loss weighting covers Uncertainty Weighting ([Kendall et al., 2018](#)), GradNorm ([Chen et al., 2018](#)), and FAMO ([B. Liu et al., 2023](#)). Gradient surgery covers PCGrad ([Yu et al., 2020](#)), CAGrad ([B. Liu et al., 2021](#)), IMTL-G ([L. Liu et al., 2021](#)), and Nash-MTL ([Navon et al., 2022](#)). The baseline group covers isolated single-task learning, uniform equal weighting, and tuned static linear scalarization. All configurations train under AdamW with FP16 mixed precision, sequence length 128, and batch size 32.

Subject 4 is the prototype triage service. The service runs as an asynchronous FastAPI application over a PostgreSQL database. The service completes inference in one pass for entities, intent, and urgency. The Philippine Standard Geographic Code (PSGC) masterlist resolves location spans, with an unmapped-span fallback. The dashboard tracks incidents across four states, namely New, Acknowledged, Responding, and Resolved. A CPU-only mode demonstrates the service on hardware without a GPU.

Table 3.1 summarizes the four subjects and their roles in the study.

Table 3.1

Summary of the Subjects of the Study

Subject	Description	Role in the Study
Multi-Task Corpus of Taglish Disaster Tweets	6,000 to 10,000 tweets from four typhoons, with language tags, BIO entity spans, intent and urgency labels	Training, validation, and test data
Encoder backbones	XLM-RoBERTa Base, multilingual BERT Base, SEA-LION ModernBERT, and RoBERTa-Tagalog Base, each with three linear heads	Shared representation learning
Optimization configurations	Seven MTO methods against STL, EW, and LS baselines	Benchmark treatments
Prototype triage service	FastAPI application, PostgreSQL incident database, PSGC gazetteer, and lifecycle dashboard	Demonstration artifact

Note. BIO = begin, inside, outside. MTO = multi-task optimization. STL = single-task learning. EW = uniform equal weighting. LS = static linear scalarization. PSGC = Philippine Standard Geographic Code.

3.3 Sampling Technique

The study samples no human participants, so sampling concerns events, tweets, and evaluation draws. The sampling technique combines purposive event selection with systematic tweet partitioning. Its purpose is to measure performance on typhoon events absent from training.

Four typhoons enter the corpus through purposive selection. The events are Haiyan, Vamco, Rai, and Paeng, the four severe Philippine typhoons that Chapter 1 selects

for documented impact and heavy social media activity. Purposive selection keeps the corpus tied to real response operations.

The tweets then undergo 4-fold LOEO partitioning. Each fold holds out one typhoon as the test set, and the remaining three typhoons supply training and validation data. A model trained on one set of typhoons therefore faces a typhoon it has never seen at test time. This design measures out-of-distribution generalization across disasters, which random splits cannot show. Table 3.2 presents the four folds.

Table 3.2

Leave-One-Event-Out Fold Composition

Fold	Test Typhoon	Training Typhoons
1	Haiyan	Vamco, Rai, and Paeng
2	Vamco	Haiyan, Rai, and Paeng
3	Rai	Haiyan, Vamco, and Paeng
4	Paeng	Haiyan, Vamco, and Rai

Note. Each held-out test set is evaluated in full, with no subsampling.

Within each fold, the three training typhoons are pooled and split once more. The split follows the HumAID development convention of a 10 percent share ([Alam et al., 2021](#)). Ten percent of the pooled tweets forms a class-stratified validation set, and the remaining 90 percent forms the training set. The validation draw is stratified by intent class so that every category reaches the validation set. The resulting ratio is reported with the results, because the exact share depends on the final corpus size.

Every configuration runs under three random seeds, 42, 123, and 456. Four folds times three seeds gives 12 trained models per optimization configuration. The seeds vary weight initialization and data order. The fold partition and the validation draw stay fixed.

The study applies no data augmentation, matching the delimitations in Chapter 1. Synthesized duplicates would also disturb the language profiles that the stratified analysis measures.

The last sampling concern is the stratified language analysis. Every test tweet receives a profile label computed from its word-level language tags. The count excludes named entities and other language-independent words, the former as a study choice and the latter per the index definition of Gambäck and Das (2016). Tweets with zero switched tokens join the English-dominant or Tagalog-dominant profile of their single language. Tweets with any switched tokens join the mixed profile. The profiles partition the held-out test set only, and the computation runs on saved predictions, so it adds no training runs.

3.4 Instruments to Be Used

This section names the instruments that execute the plan of Sections 3.2 and 3.3. They cover model training, gradient diagnostics, performance profiling, run logging, statistics, and annotation.

Model implementation and training run on PyTorch with the Hugging Face Transformers library. NumPy and pandas handle data manipulation. The class-stratified validation draws of Section 3.3 execute through scikit-learn.

The gradient diagnostics are custom PyTorch hooks registered on the shared backbone parameters. At every training step, the hooks record per-task gradient norms and pairwise cosine similarities. The intra-task half-batch baseline of Elich et al. (2024) separates stochastic noise from genuine conflict. For this baseline, each batch splits into halves, the task gradients are computed per half, and their similarities form the noise

reference.

Latency and throughput timing uses high-resolution timers with warm-up runs and repeated measurements, and the reported value is the median across repetitions. The PyTorch Profiler traces compute, pynvml reads GPU memory, and psutil reads host RAM.

Every run logs to a local MLflow tracking server. A run record carries the configuration, fold, seed, metric curves, and hardware readings. It also stores the exact library versions, so every results table derives from a reproducible run.

Statistical testing uses SciPy for the Friedman test and the Spearman correlation, plus a custom paired bootstrap procedure with 10,000 resamples. The Friedman test and its Nemenyi post-hoc procedure follow the two-tier protocol of Demšar (2006). The formulas appear in Section 3.6.

Annotation runs on Label Studio, a self-hosted labeling tool that keeps tweet content on local infrastructure. The tool supports word-level labels for the language tags and the BIO spans, and its review workflow implements the adjudication step of Section 3.5.

Functional verification uses pytest for the unit and integration tests planned in Section 3.9. Table 3.3 consolidates the instruments, their roles, and their versions.

Table 3.3

Summary of Instruments

Instrument	Role in the Study	Version
PyTorch	Model implementation, training loops, and gradient hooks	Pinned at implementation
Hugging Face Transformers	Encoder backbones and task heads	Pinned at implementation

Instrument	Role in the Study	Version
scikit-learn	Class-stratified validation draws and label encoding	Pinned at implementation
NumPy and pandas	Data manipulation	Pinned at implementation
MLflow, local server	Run tracking for parameters, metrics, and versions	Pinned at implementation
PyTorch Profiler	Compute tracing	Pinned at implementation
pynvml	GPU memory readings	Pinned at implementation
psutil	Host RAM readings	Pinned at implementation
SciPy	Friedman test and Spearman correlation	Pinned at implementation
Custom bootstrap script	Paired bootstrap with 10,000 resamples	Repository code
Label Studio	Annotation and adjudication workflow	Pinned at implementation
pytest	Unit and integration tests	Pinned at implementation

Note. The development environment runs Python 3.12 on Linux. Version cells hold the release targeted at implementation, and every MLflow run also stores the exact library versions for reporting in the results chapter. The workstation testbed itself appears in Section 3.8.3.

3.5 Procedure of Data Gathering

Section 3.4 named the instruments. This section orders them into the following eight stages. Stages 1 to 6 construct the corpus artifact, and stages 7 to 8 execute the benchmarking plan. The stages run in sequence, and every stage ends with a defined output.

1. **Acquisition.** The study collects historical public posts from Twitter/X for the four typhoons of Section 3.2. Retrieval runs through the paid full-archive search of the platform, which bills per retrieved post after the end of free academic access in

2023. For Haiyan, the study seeds the retrieval with the tweet identifiers of the CrisisNLP corpora (Imran et al., 2016), so the paid reads cover only known-relevant posts. The three newer typhoons enter through direct keyword retrieval, because the established crisis corpora end before their events. The queries combine event names with civic hashtags such as #RescuePH and #ReliefPH (Ermino et al., 2022). Each query runs twice, once with English and once with Filipino as the query language, following the search design of Herrera et al. (2022). The collection window of each typhoon covers the landfall period and the response weeks that follow. Some older identifiers no longer resolve after account deletion, so the study reports the retrieval yield per event. Each stored record keeps the tweet identifier, the timestamp, and the raw text.

2. **Filtering and deduplication.** The pipeline removes retweets and exact duplicates, and it detects near-duplicates by comparing lowercased text without punctuation, URLs, and hashtags. This screening follows the duplicate-free requirement of HumAID (Alam et al., 2021). Tweets with fewer than three words are removed under the same rule. Tweets that contain neither Tagalog nor English text are also removed, because the delimitations confine the corpus to the Taglish register. Stages 1 and 2 repeat until the corpus reaches at least 6,000 verified tweets, the minimum of Section 3.2.

3. **Language tagging.** The same three annotators serve stages 3 to 5 and work independently throughout. Every word receives one of five tags, English, Tagalog, mixed, ambiguous, or other, following the word-level tagset that Gambäck and Das

(2016) describes. A word that joins both languages in one word form, such as “ma-evacuate”, receives the mixed tag. A word that can belong to either language even in context receives the ambiguous tag. Abbreviations take the tag of the language they belong to, and numbers, emoticons, and symbols receive the tag other.

4. **Annotation.** A codebook defines every label with decision rules and worked examples. The codebook maps each of the 11 HumAID categories to its merged triage class (Alam et al., 2021) and restates the four-level urgency scale of Sarioglu Kayi et al. (2020). The annotators label intent and urgency per tweet and draw BIO entity spans per word in Label Studio. Each tweet receives three independent judgments for these labels, following the HumAID protocol (Alam et al., 2021).
5. **Agreement measurement.** A tweet label is approved when two of the three judgments agree, following the HumAID approval rule (Alam et al., 2021). Word-level tags and BIO spans are approved by per-token majority, and tokens without a majority enter adjudication. The study computes Fleiss’ kappa per layer and reads each value against the bands of Landis and Koch (1977). The moderate band is the minimum acceptable level, and a layer below it triggers a codebook revision and re-annotation of the affected tweets. Cases that majority voting cannot settle enter the Label Studio review workflow, and the three annotators decide the final label together.
6. **Preprocessing and alignment.** Approved tweets are serialized into the JSONL schema of Section 3.2, one object per tweet with all seven fields. URLs and user mentions become fixed placeholders, and the remaining spelling and casing stay

untouched. The alignment rule copies the label of a word to its first subword token, and the remaining subwords carry an ignore marker, so the loss counts every word once. The pipeline applies the rule to the tokenizer of the selected backbone before training starts. Tweets longer than the 128-token limit are truncated, and an automated pytest case verifies the alignment of every tweet. Any public release shares only annotations against tweet identifiers, which follows the release practice of TweetTaglish ([Herrera et al., 2022](#)).

7. **Training protocol.** Training begins with the encoder selection of Section 3.2. The four candidates undergo fine-tuning under uniform equal weighting, and the study keeps the backbone with the highest mean macro F1. The full benchmark then trains the 10 configurations under every fold and seed combination, for 120 runs. Every run uses AdamW, FP16 mixed precision, sequence length 128, and batch size 32. A single learning rate, selected from the fine-tuning range of Devlin et al. ([2019](#)), serves every configuration, and early stopping monitors validation macro F1. The gradient hooks of Section 3.4 record norms and cosines at every step, and MLflow stores the complete run record.
8. **Efficiency measurement.** The study measures the joint model of the selected backbone and the cascade of three single-task models on CPU hardware, the deployment condition that Chapter 1 motivates. Warm-up runs load the weights into memory and fill the processor caches before timing starts. Latency is measured one tweet at a time, and throughput is measured over batches of 32. The reported values are medians across repetitions, and the PyTorch Profiler and psutil supply the traces

and the memory readings.

Sections 3.6 and 3.7 define the statistics and the analysis applied to these outputs.

3.6 Statistical Treatment

Section 3.5 produced the run records and the efficiency readings. This section defines the inferential tests that decide the research questions. The treatment follows the two-tier nonparametric protocol of Demšar (2006), whose first tier performs targeted pairwise tests and whose second tier performs an omnibus comparison with a post-hoc procedure. The significance level is 0.05 for every test. The protocol is nonparametric because the runs form small dependent samples, for which rank-based tests are the appropriate choice (Demšar, 2006).

The treatment states 14 hypotheses in seven pairs. Every odd-numbered hypothesis is a null hypothesis, and every even-numbered hypothesis is its logical negation. The pairs map onto the specific problems of Chapter 1.

1. **H01 and H02 cover Specific Problem 1.** H01 states that the joint encoder under uniform equal weighting and the isolated single-task models do not differ in per-task F1 on the held-out folds.
2. **H03 and H04 cover intent performance.** H03 states that the 10 configurations do not differ in intent macro F1.
3. **H05 and H06 cover urgency performance.** H05 states that the 10 configurations do not differ in urgency macro F1.
4. **H07 and H08 cover entity performance.** H07 states that the 10 configurations do not differ in span-level entity F1.

5. **H09 and H10 cover inference efficiency.** H09 states that the joint encoder and the single-task cascade do not differ in latency, throughput, or memory footprint.
6. **H11 and H12 cover transfer and conflict.** H11 states that relative multi-task transfer Δ_m and the gradient conflict readings of Section 3.4 have no monotone correlation.
7. **H13 and H14 cover the language profiles.** H13 states that the configuration ranking stays identical across the three profiles of Section 3.3.

Specific Problem 2 receives no pair, because it concerns artifact design, and Section 3.9 tests its acceptance directly. The Friedman and Nemenyi procedures evaluate the omnibus claims in H03, H05, and H07. The paired bootstrap evaluates the pairwise claims inside H01 to H10, including each comparison against the STL, EW, and LS baselines. The Spearman correlation evaluates H11 and H12. Friedman comparisons within the language profiles evaluate H13 and H14.

The paired bootstrap compares two configurations on one metric within one fold. The two configurations label the same held-out test set of Section 3.3, and a resample draws the same number of instances with replacement. The resampled statistic is the metric difference

$$d^* = \hat{m}_a(S^*) - \hat{m}_b(S^*)$$

where $\hat{m}_a$ and $\hat{m}_b$ are the metric values of the two configurations on a resample S^* .

The procedure draws 10,000 resamples and sorts the resulting differences. The 95 percent confidence interval runs from the 2.5th to the 97.5th percentile. A difference is declared

significant when the interval excludes zero, and one interval is reported per fold. The efficiency claim in H09 uses the same procedure on run-level readings. The paired observations are the 12 fold-seed runs of Section 3.3, and the statistic is the mean paired difference

$$\bar{d} = \frac{1}{n} \sum_{i=1}^n \left(y_i^{(a)} - y_i^{(b)} \right), \quad n = 12$$

where $y_i^{(a)}$ and $y_i^{(b)}$ are the latency, throughput, or memory readings of the two systems on run i .

The Friedman test compares k configurations over N repeated blocks, and the study applies the Iman-Davenport correction for the small block count ([Demšar, 2006](#)). The primary blocks are the 12 fold-seed runs. The sensitivity analysis averages the three seeds within each fold, which gives $N = 4$ and matches the sensitivity check of Chapter 1. Let R_j denote the mean rank of configuration j over the blocks. The corrected statistic follows an F distribution with $k - 1$ and $(N - 1)(k - 1)$ degrees of freedom.

$$F_F = \frac{(N - 1) \chi_F^2}{N(k - 1) - \chi_F^2}, \quad \chi_F^2 = \frac{12N}{k(k + 1)} \left[\sum_{j=1}^k R_j^2 - \frac{k(k + 1)^2}{4} \right]$$

The omnibus claim is rejected when F_F exceeds the critical F value at the 0.05 level. A rejected omnibus test triggers the Nemenyi procedure, which compares every configuration pair through their mean rank difference ([Demšar, 2006](#)). Two configurations differ significantly when their rank difference exceeds the critical difference

$$CD = q_\alpha \sqrt{\frac{k(k+1)}{6N}}$$

where q_α is the critical value of the Studentized range statistic divided by $\sqrt{2}$, taken at $\alpha = 0.05$.

The Spearman correlation evaluates the association between relative multi-task transfer Δ_m of Chapter 1 and the mean pairwise gradient cosine of Section 3.4. The computation runs over the 12 fold-seed runs, with averaged ranks for ties. With rank differences d_i , the correlation is

$$\rho = 1 - \frac{6 \sum_{i=1}^n d_i^2}{n(n^2 - 1)}, \quad n = 12$$

The correlation is significant when its p-value falls below 0.05. Section 3.7 defines the descriptive measures and the per-task metric formulas that enter these tests.

3.7 Data Analysis

Section 3.6 defined the tests. This section defines the measures that enter them. Every measure is computed from the saved predictions and run records of each fold-seed run, so the analysis adds no training runs. Each paragraph names one measure, states its equation, and identifies the research question it serves.

The sequence tasks are scored by macro-averaged F1, which is the unweighted mean of class-level F1 over the classes of one task. With class precision P_c and recall R_c , the macro score is

$$\text{Macro F1} = \frac{1}{|C|} \sum_{c=1}^{|C|} \frac{2P_c R_c}{P_c + R_c}$$

The intent score averages the merged triage classes, and the urgency score averages the two classes of the binary conversion of Section 3.2. The entity task is scored by span-level F1 under the CoNLL standard. A predicted span counts as correct only when its boundaries and its category match a gold span.

$$\text{Span F1} = \frac{2P_{\text{span}} R_{\text{span}}}{P_{\text{span}} + R_{\text{span}}}$$

These three scores answer the performance question of Specific Problems 1 and 3, and they feed the hypothesis pairs H01 to H08.

Relative multi-task transfer compares each configuration against the isolated single-task models of the same fold and seed. Chapter 1 defines the composite measure, and the notation follows Chapter 1.

$$\Delta_m = \frac{1}{3} \sum_{k=1}^3 \frac{\text{F1}_{\text{MTL},k} - \text{F1}_{\text{STL},k}}{\text{F1}_{\text{STL},k}} \times 100\%$$

Here k ranges over the three tasks. Positive values indicate positive transfer, and negative values quantify net negative transfer. The single-task configuration yields zero transfer by construction. This measure answers the transfer component of Specific Problem 3 and enters the correlation test of H11 and H12.

The gradient diagnostics quantify interference on the shared parameters. At every training step, the hooks record for each task pair the cosine between their gradients,

$$\cos(\mathbf{g}_i, \mathbf{g}_j) = \frac{\langle \mathbf{g}_i, \mathbf{g}_j \rangle}{\|\mathbf{g}_i\|_2 \|\mathbf{g}_j\|_2}$$

and the norm disparity ratio of Chapter 1,

$$R_{\max/\min}^{(t)} = \frac{\max_k \|\mathbf{g}_k^{(t)}\|_2}{\min_k \|\mathbf{g}_k^{(t)}\|_2}$$

Both statistics are also computed within one task across batch halves, which forms the intra-task noise reference of Elich et al. (2024). A pairwise cosine below its within-task reference marks genuine conflict rather than stochastic noise. These measures answer the interference question of Specific Problems 3 and 4, and their run-level means enter the correlation test of H11 and H12.

The code-mixing index quantifies the degree of switching in one tweet from its word-level tags. The study uses the weighted utterance measure of Gambäck and Das (2016),

$$C_u = 100 \times \frac{w_m [N - \max_{L_i \in \mathcal{L}} t_{L_i}] + w_p P}{N}$$

Here N is the token count of the tweet, and t_{L_i} is the count of tokens tagged with language L_i . P is the number of alternation points, and w_m and w_p are weights with a sum of one. The study sets both weights to one half. The analysis averages the index over tweets, following the same source,

$$C_{\text{avg}} = \frac{1}{U} \sum_{u=1}^U C_u$$

where U is the number of tweets in the group. The index answers the code-switching question of Specific Problem 4 as a descriptive companion to the stratified comparison. The profile assignment itself follows the rule of Section 3.3 and stays separate from the index value.

Subword fertility measures tokenizer strain on Taglish text. For a word w segmented into s_w subword tokens by the selected tokenizer, the fertility over a group of tweets is

$$f = \frac{1}{|W|} \sum_{w \in W} s_w$$

where W is the set of words in the group. The fragmentation comparison follows the practice of Petrov et al. (2023). The computation runs per language profile of Section 3.3 on saved predictions. This measure answers the code-switching question of Specific Problem 4, because tokenization strain helps explain per-profile behavior.

Latency, throughput, and memory are computed from the timing stage of Section 3.5. The reported latency is the median across timed repetitions after warm-up,

$$\hat{t}_{\text{exec}} = \text{median}\{\tau_1, \tau_2, \dots, \tau_R\}$$

where τ_r is the elapsed time of repetition r . Throughput divides the batch size by the median batch time, and memory takes the maximum reading within the inference window,

$$\text{TP} = \frac{B}{\text{median}\{\tau_r^{\text{batch}}\}}, \quad M = \max_r m_r$$

Latency is read at batch size 1, and throughput is read at the batch size of 32, matching Chapter 1. Memory covers peak GPU memory and host RAM through the readings of Section 3.4. These measures answer the efficiency question of Specific Problem 3 and feed the hypothesis pair H09 and H10.

Together these measures turn the saved predictions and run records into the tables of the results chapter. Section 3.8 describes the software system that houses the serving path under measurement.

3.8 Software System Description

Sections 3.2 to 3.7 defined the subjects, the sampling plan, the instruments, the procedure, and the analysis. This section describes the software artifact that realizes them. The artifact spans two parts, the training and benchmarking pipeline, and the prototype triage service. Both parts live as versioned repository code, and every component is regenerable from its source.

3.8.1 System Structure

Figure 3.1 presents the end-to-end pipeline of the study under the four DSRM phases. The Baseline Analysis phase assembles the corpus through the stages of Section 3.5 and measures the isolated single-task baselines. The Design and Artifact Creation phase implements the hard-sharing trainer, the optimization configurations, the gradient hooks, and the prototype service. The Empirical Benchmarking phase executes the 120 runs of Section 3.5 and records each run in MLflow. The Comparative Validation phase

applies the tests of Section 3.6 to the saved outputs.

Figure 3.1

End-to-End Pipeline of the Study

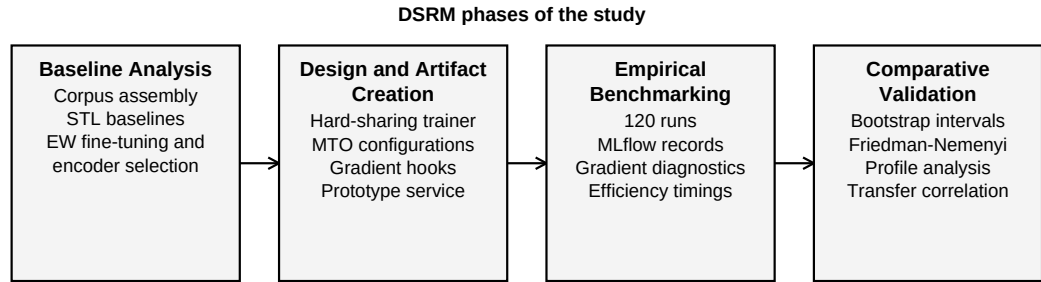


Figure 3.2 presents the model layer. The tokenizer converts the raw tweet into subword tokens, and the shared multilingual encoder computes one contextual representation per token. Three linear heads read that representation, one token head for BIO entity labels, one sequence head for intent, and one sequence head for urgency. The encoder runs once per tweet, so the inference cost stays close to single-encoder cost.

Figure 3.2

Model Layer Flow under Hard Parameter Sharing

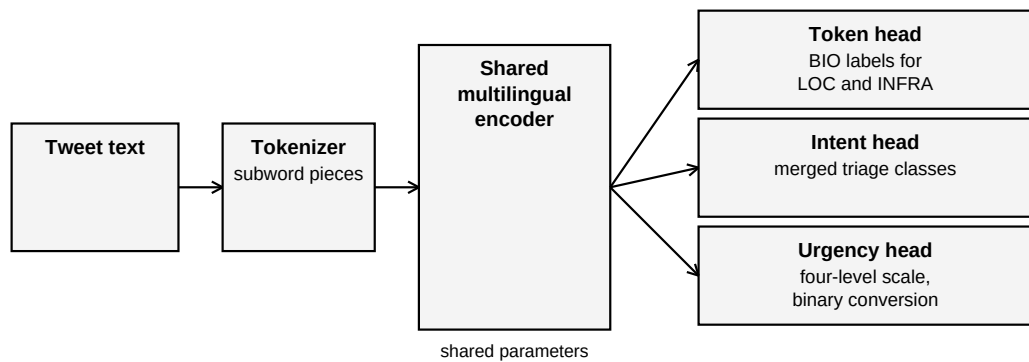
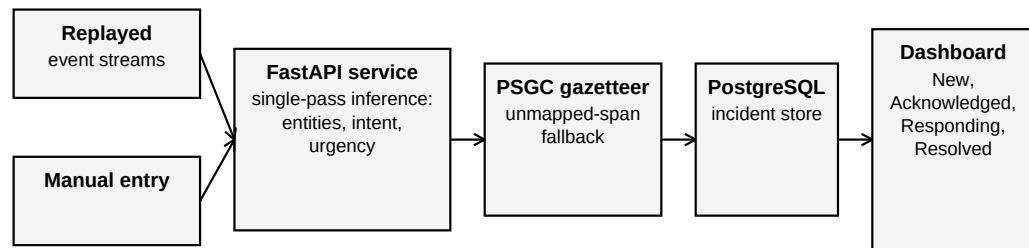


Figure 3.3 presents the data flow of the prototype service. Replayed event streams and manual entry feed Taglish messages to the asynchronous FastAPI service. The service completes single-pass inference for entities, intent, and urgency, and resolves location

spans against the PSGC masterlist with an unmapped-span fallback. Recognized incidents persist in the PostgreSQL database, and the dashboard tracks each incident across the four lifecycle states of Subject 4.

Figure 3.3

Data Flow of the Prototype Triage Service

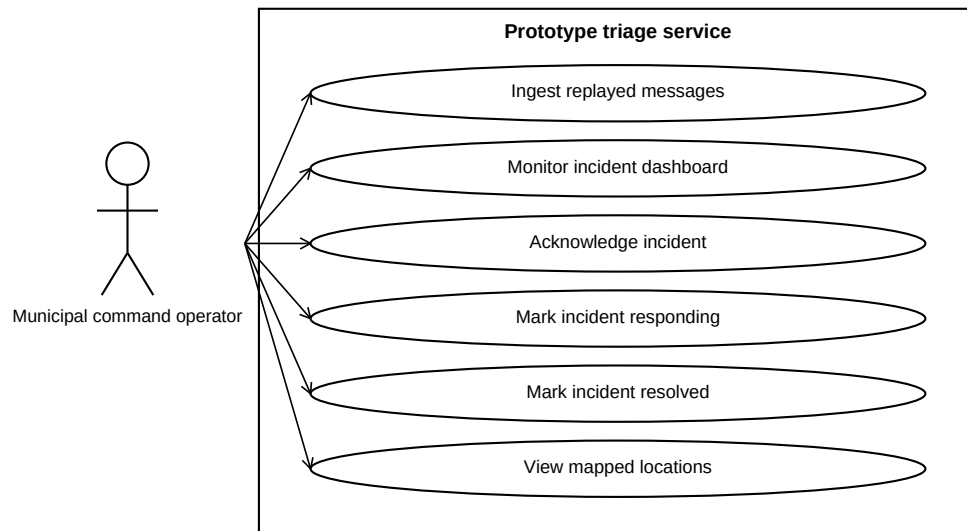


3.8.2 Use Case Model

The use case model covers the prototype service from the operator's perspective. Figure 3.4 presents the use case diagram. One primary actor, the municipal command operator, works inside the system boundary of the prototype. The operator ingests replayed messages, monitors the incident dashboard, advances an incident through its lifecycle states, and views the mapped locations of extracted entities. The three tables below specify the ingestion, monitoring, and state-update use cases.

Figure 3.4

Use Case Diagram of the Prototype Triage Service

**Table 3.4***Use Case Specification for Message Ingestion*

Field	Specification
Use case	Ingest replayed messages
Actor	Municipal command operator
Precondition	A replay archive for one typhoon exists on the local server
Main flow	1. The operator selects a replay archive. 2. The service streams the messages one at a time. 3. The service stores each inference result as an incident.
Alternate flow	An unreadable record is skipped and logged with a failure entry.
Postcondition	The new incidents appear on the dashboard in the New state.

Table 3.5*Use Case Specification for Dashboard Monitoring*

Field	Specification
Use case	Monitor the incident dashboard
Actor	Municipal command operator

Field	Specification
Precondition	At least one incident exists in the database
Main flow	1. The operator opens the dashboard. 2. The dashboard groups the incidents by lifecycle state. 3. The operator opens one incident and reads its entities, intent, urgency, and mapped location.
Alternate flow	A location span has no PSGC match, and the incident is marked as unmapped.
Postcondition	The dashboard shows the current state of every incident.

Table 3.6*Use Case Specification for State Updates*

Field	Specification
Use case	Update an incident state
Actor	Municipal command operator
Precondition	An incident exists in the New, Acknowledged, or Responding state
Main flow	1. The operator opens one incident. 2. The operator selects the next state in the sequence. 3. The service records the change with a timestamp.
Alternate flow	A transition that skips the state sequence is rejected, and the incident keeps its current state.
Postcondition	The incident carries its new state and update timestamp.

3.8.3 Software and Hardware Specifications

The deployment stack runs on Python 3.12 on Linux, matching the environment note of Table 3.3. The service uses FastAPI, and PostgreSQL stores the incident records. Every library version is pinned at implementation, and every MLflow run stores the exact versions, as Section 3.4 specifies.

The service runs fully offline. The model weights, the tokenizer, the PSGC masterlist, and the database live on the local machine, and the inference path calls no external service. Local operation keeps citizen messages on the infrastructure of the responding institution, which supports the compliance obligations of the Data Privacy Act of 2012 ([Republic of the Philippines, 2012](#)).

The training testbed is a single-GPU workstation that supports FP16 mixed precision, which Subject 3 requires. The prototype also runs in a CPU-only mode on hardware without a GPU, which demonstrates the deployment constraint of Chapter 1 and prepares the CPU stress test of Section 3.9.

3.9 Testing and Evaluation

This section closes the chapter with the testing protocol for the software artifact and the evaluation plan for the completed runs. The acceptance of Specific Problem 2 rests on these tests, as Section 3.6 states. All tests execute through pytest, as Section 3.4 specifies, and a failure in any test group blocks the benchmark matrix until it is resolved.

The unit tests target the small mathematical components. The alignment tests verify the first-subword rule of Section 3.5 for every tweet in the corpus. The balancing tests verify the optimization implementations against hand-computed examples on tiny inputs. The projected gradient of PCGrad, for example, must be orthogonal to the conflicting gradient after projection ([Yu et al., 2020](#)). The hook tests verify that the cosine matrix is symmetric with a unit diagonal, and the index tests verify that a monolingual tweet yields a code-mixing index of zero. The state machine tests verify the lifecycle transitions of Table 3.6.

The determinism checks run one configuration twice under the same seed, the same fold partition, and the same data order. The two runs must produce identical metric curves within a fixed floating-point tolerance, and MLflow stores both runs for comparison. A mismatch indicates undeclared randomness in the pipeline.

The integration tests exercise the full pipeline on a miniature sample. A small annotated subset passes through training, inference, gazetteer resolution, incident storage, and the dashboard transitions inside one test session. One test submits a message to the FastAPI service and verifies the incident in PostgreSQL with its entities, intent, urgency, and lifecycle state. Another test feeds a location span with no PSGC match and verifies the unmapped-span fallback of Section 3.8. Table 3.7 consolidates the test groups and their passing criteria.

Table 3.7

Test Groups and Passing Criteria

Test group	Scope	Passing criterion
Alignment tests	First-subword rule on every corpus tweet	Every label lands on the first subword of its word
Balancing tests	Optimization implementations on tiny inputs	Outputs match hand-computed values
Hook tests	Gradient norms and cosines	Symmetric cosine matrix with a unit diagonal
Index tests	Code-mixing index computation	Monolingual tweets yield an index of zero
State machine tests	Incident lifecycle transitions	Only valid transitions are accepted
Determinism checks	One configuration run twice	Identical metric curves within tolerance
Integration tests	End-to-end pipeline on a miniature sample	Training, inference, and storage complete in one session

Test group	Scope	Passing criterion
------------	-------	-------------------

Note. All groups execute through pytest on every change to the repository code.

The evaluation then executes the full benchmark. The encoder selection stage trains the four candidates under uniform equal weighting across the four folds and three seeds, for 48 runs. The benchmark stage trains the 10 configurations of Section 3.2 under every fold and seed combination, for the 120 runs of Section 3.5. Every run logs to MLflow with its configuration, fold, seed, metric curves, gradient diagnostics, and exact library versions.

The CPU stress test evaluates the prototype under sustained load. The CPU-only mode ingests a full replayed fold at peak rate while the timing stage of Section 3.5 records latency, throughput, and memory readings. The test verifies that the service stays responsive on hardware without a GPU, which addresses the deployment constraint of Chapter 1.

The reproducibility rerun closes the evaluation. One full configuration is rerun across all folds and seeds, for 12 runs, after the environment is re-created from the logged versions. The rerun metrics must match the originals within the same tolerance as the determinism checks. Together the matrix, the stress test, and the rerun produce the evidence that Sections 3.6 and 3.7 analyze.

References

Alam, F., Qazi, U., Imran, M., & Ofli, F. (2021). HumAID: Human-annotated disaster incidents data from Twitter with deep learning benchmarks. *Proceedings of the*

International AAAI Conference on Web and Social Media (ICWSM 2021).

<https://doi.org/10.1609/icwsml.v15i1.18116>

Chen, Z., Badrinarayanan, V., Lee, C.-Y., & Rabinovich, A. (2018). GradNorm: Gradient normalization for adaptive loss balancing in deep multitask networks. *Proceedings of the 35th International Conference on Machine Learning (ICML 2018).*

<https://arxiv.org/abs/1711.02257>

Demšar, J. (2006). Statistical comparisons of classifiers over multiple data sets. *Journal of Machine Learning Research*. <https://www.jmlr.org/papers/v7/demsar06a.html>

Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2019). BERT: Pre-training of deep bidirectional transformers for language understanding. *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT 2019).*

<https://doi.org/10.18653/v1/N19-1423>

Elich, C., Kirchdorfer, L., Köhler, J. M., & Schott, L. (2024). Examining common paradigms in multi-task learning. *46th DAGM German Conference on Pattern Recognition (GCPR 2024)*. https://doi.org/10.1007/978-3-031-85181-0/_9

Ermino, R. P., Abacan, R. C. A., Diamante, N. G. C., Faca, K. M. N., & Juntereal, T. E. D. (2022). Evaluating urgency of Typhoon-Related tweets through sentiment analysis using artificial neural networks. *International Journal of Computing Sciences Research*. <https://doi.org/10.25147/ijcsr.2017.001.1.80>

Gambäck, B., & Das, A. (2016). Comparing the level of code-switching in corpora. *Proceedings of the Tenth International Conference on Language Resources and Evaluation (LREC 2016)*. <https://aclanthology.org/L16-1292/>

- Herrera, M., Aich, A., & Parde, N. (2022). TweetTaglish: A dataset for investigating Tagalog-English code-switching. *Proceedings of the Thirteenth Language Resources and Evaluation Conference*. <https://aclanthology.org/2022.lrec-1.225>
- Hevner, A. R., Ram, S., March, S. T., & Park, J. (2004). Design science in information systems research. *MIS Quarterly*. <https://doi.org/10.2307/25148625>
- Imran, M., Mitra, P., & Castillo, C. (2016). Twitter as a lifeline: Human-annotated Twitter corpora for NLP of crisis-related messages. *Proceedings of the Tenth International Conference on Language Resources and Evaluation (LREC 2016)*. <https://doi.org/10.63317/43bfut6ipc3j>
- Kendall, A., Gal, Y., & Cipolla, R. (2018). Multi-task learning using uncertainty to weigh losses for scene geometry and semantics. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR 2018)*. <https://doi.org/10.1109/CVPR.2018.00781>
- Landis, J. R., & Koch, G. G. (1977). The measurement of observer agreement for categorical data. *Biometrics*. <https://doi.org/10.2307/2529310>
- Liu, B., Feng, Y., Stone, P., & Liu, Q. (2023). FAMO: Fast adaptive multitask optimization. *Advances in Neural Information Processing Systems 36 (NeurIPS 2023)*. <https://arxiv.org/abs/2306.03792>
- Liu, B., Liu, X., Jin, X., Stone, P., & Liu, Q. (2021). Conflict-averse gradient descent for multi-task learning. *Advances in Neural Information Processing Systems 34 (NeurIPS 2021)*. <https://arxiv.org/abs/2110.14048>
- Liu, L., Li, Y., Kuang, Z., Xue, J.-H., Chen, Y., Yang, W., Liao, Q., & Zhang, W. (2021). TOWARDS IMPARTIAL MULTI-TASK LEARNING. *International Conference*

- on Learning Representations*. <https://openreview.net/forum?id=IMPnRXEWpvr>
- Navon, A., Shamsian, A., Achituve, I., Maron, H., Kawaguchi, K., Chechik, G., & Fetaya, E. (2022). Multi-task learning as a bargaining game. *International Conference on Machine Learning (ICML 2022)*. <https://arxiv.org/abs/2202.01017>
- Peffer, K., Tuunanen, T., Rothenberger, M. A., & Chatterjee, S. (2007). A design science research methodology for information systems research. *Journal of Management Information Systems*. <https://doi.org/10.2753/MIS0742-1222240302>
- Petrov, A., La Malfa, E., Torr, P. H. S., & Bibi, A. (2023). Language model tokenizers introduce unfairness between languages. *Advances in Neural Information Processing Systems 36 (NeurIPS 2023)*. <https://arxiv.org/abs/2305.15425>
- Republic of the Philippines. (2012). *Republic act no. 10173: Data privacy act of 2012*. Official Gazette. <https://www.officialgazette.gov.ph/2012/08/15/republic-act-no-10173/>
- Sarioglu Kayi, E., Nan, L., Qu, B., Diab, M., & McKeown, K. (2020). Detecting urgency status of crisis tweets: A transfer learning approach for low resource languages. *Proceedings of the 28th International Conference on Computational Linguistics*. <https://aclanthology.org/2020.coling-main.414>
- Yu, T., Kumar, S., Gupta, A., Levine, S., Hausman, K., & Finn, C. (2020). Gradient surgery for multi-task learning. *Advances in Neural Information Processing Systems 33 (NeurIPS 2020)*. <https://arxiv.org/abs/2001.06782>